

FACULDADE ANHANGUERA

Tecnologia em Análise e Desenvolvimento de Sistemas

Projeto Integrado I, II e III

Gabriel Rodrigues do Prado - 3º semestre

Rafaela Rodrigues Oliveira - 1º semestre

Brasa Burger Management

**Proposta e modelagem integrada de um sistema web de gestão multiunidade para
food service**

Trabalho parcial apresentado ao Projeto Integrado I, II e III, referente à Entrega 1 do período letivo 2026/2. Docente: Prof. Gilberto Falco Netto.

Jundiaí

13 de setembro de 2026

Sumário

1 Introdução e objetivos	3
2 Escopo e continuidade	4
3 Arquitetura e tecnologias	5
4 Matriz de integração curricular	6
5 Modelagem algorítmica	7
6 Modelagem matemática	8
7 Análise orientada a objetos	9
8 Banco de dados e camada analítica	10
9 Computação em nuvem e segurança	11
10 Plano de implementação e validação	12
11 Repositório e organização da equipe	13
12 Perspectivas e considerações finais	14
Referências	15

Curso: Tecnologia em Análise e Desenvolvimento de Sistemas. Disciplina integradora: Projeto Integrado I, II e III. Período letivo: 2026/2. Integrantes: Gabriel Rodrigues do Prado, 3º semestre, e Rafaela Rodrigues Oliveira, 1º semestre.

1 Introdução e objetivos

O Brasa Burguer Management é um projeto acadêmico de sistema web para gestão de hamburguerias e pequenas redes de food service. A proposta conecta clientes, produtos, pedidos, fichas técnicas, estoque e indicadores em uma operação multiunidade. O caso de uso considera uma rede fictícia com as unidades Japy, Retiro e Eloy, em Jundiá, e permite analisar tanto uma loja quanto o conjunto da operação.

O problema tratado é a fragmentação das informações entre atendimento, produção e gestão. Um pedido contém dados úteis sobre o cliente, os produtos escolhidos, o canal de venda, a unidade e os custos envolvidos. Quando esses dados ficam em controles separados, o operador repete cadastros e o gestor encontra dificuldades para recuperar históricos e conferir resultados.

Os interessados são operadores de atendimento, gerentes, responsáveis por estoque e administradores da rede. A relevância acadêmica está na aplicação de programação e modelagem a um domínio compreensível, no qual uma ação comercial produz efeitos operacionais e informações para análise.

1.1 Objetivo geral

Desenvolver e documentar a evolução de uma aplicação web de gestão multiunidade, relacionando o registro de pedidos ao consumo de ingredientes e aos indicadores gerenciais, com interface em React e serviços de dados e autenticação no Supabase.

1.2 Objetivos específicos

- Organizar o fluxo de cadastro e identificação do cliente durante a venda.
- Relacionar produtos, variações, fichas técnicas e consumo por unidade.
- Explicar e validar os cálculos de custo, CMV, margem e ticket médio.
- Representar o domínio por objetos e decompor as regras em procedimentos verificáveis.
- Formalizar a arquitetura em nuvem, o versionamento e um plano de testes compatível com a Entrega 1.

1.3 Método de desenvolvimento

O trabalho adota desenvolvimento incremental: delimitação de uma melhoria, modelagem, implementação, teste e registro da evidência antes da etapa seguinte. A ficha técnica e a apresentação do projeto descrevem o estado funcional. A validação executável deverá vincular essa descrição ao código e às migrações da aplicação.

2 Escopo e continuidade

A modalidade é continuidade de um protótipo acadêmico existente. O termo linha de base identifica o estado recebido no início desta revisão. Não se atribui uma funcionalidade a um semestre anterior sem registro que comprove sua data. Para a avaliação de 2026/2, o grupo deverá associar cada incremento aos commits, às decisões e aos artefatos efetivamente produzidos ou revistos.

Base anteriormente apresentada	Estado descrito nos materiais atualizados
Cadastros de clientes e produtos	Perfil do cliente, histórico e cardápio com variações
Tela de pedidos e painel inicial	Kanban, canais, pagamentos e filtros
Entidades relacionais e consultas	Funções, triggers, views e fundação analítica
Aplicação em localhost	React/Vite conectado ao Supabase

2.1 Funcionalidades descritas na versão atual

A apresentação atualizada informa login com sete perfis, gestão de clientes, histórico de pedidos, cardápio, adicionais, fichas técnicas, baixa automática de estoque, indicadores e visão de três lojas. A ficha técnica detalha as tecnologias e automações que sustentam essas funcionalidades. Essa descrição documental não substitui testes de execução e autorização.

2.2 Delimitação da Entrega 1

A primeira entrega concentra problema, objetivos, escopo, continuidade, integração curricular, modelagem, plano de implementação e evidência inicial no repositório. A amplitude do produto não exige concluir todos os módulos de um ERP antes da avaliação. O núcleo escolhido para explicação é o pedido e seus efeitos sobre custo e estoque.

2.3 Evoluções posteriores

Permanecem como evolução a atualização do custo dos ingredientes a partir de compras reais, o financeiro, a conexão efetiva com Power BI, integrações com marketplaces e meios de pagamento, emissão fiscal, SaaS comercial e recursos de inteligência artificial. O cadastro de fornecedores existente não equivale a um processo completo de compras integrado.

A hospedagem do front-end no Netlify é planejada. A ficha técnica informa que o Git, os testes automatizados e o fluxo de integração e publicação contínuas ainda precisam ser formalizados.

3 Arquitetura e tecnologias

A aplicação utiliza Backend as a Service (BaaS). O navegador executa a interface React e acessa o Supabase por uma API REST gerada pelo PostgREST a partir do schema do PostgreSQL. A documentação do Supabase descreve esse acesso direto pelo navegador, com integração ao modelo de permissões do banco (SUPABASE, s.d.a). Não há servidor de aplicação próprio descrito na arquitetura atual.

Camada	Tecnologia	Papel no projeto
Interface	React 18 e JSX	Componentes e interação com o usuário
Desenvolvimento	Vite e npm	Servidor local, dependências e build
Estilo e navegação	Tailwind CSS e React Router	Estilo consistente e navegação SPA
Gráficos	Recharts	Representação dos indicadores
API e identidade	PostgREST e Supabase Auth	CRUD, login e sessão
Regras e dados	PostgreSQL, SQL e PL/pgSQL	Tabelas, views, funções e triggers
Automação externa	Python 3	Seeds e simulações de pedidos

3.1 Responsabilidade de cada linguagem

JavaScript/JSX compõe a interface, a lógica de interação e as chamadas à API. SQL define estruturas e consultas; PL/pgSQL permite organizar procedimentos no banco. HTML e CSS aparecem na renderização da interface e no uso de JSX e Tailwind. A divisão em componentes ajuda a organizar estados e fluxo de dados no React (REACT, s.d.).

Python executa scripts independentes, como `seed_data.py` e `seed_cardapio.py`, que preparam cardápio e dados de teste. Ele não funciona como uma API intermediária obrigatória entre React e Supabase. Node.js pode participar do ambiente de ferramentas, mas isso não caracteriza um backend próprio em Node.js.

O MySQL mencionado na ficha pertence a um ambiente separado de estudo de SQL analítico. O banco transacional da aplicação é PostgreSQL no Supabase.

4 Matriz de integração curricular

O grupo reúne integrantes de semestres diferentes. Gabriel cursa Desenvolvimento em JavaScript, Computação em Nuvem e Análise Orientada a Objetos. A disciplina de algoritmos associada à linguagem C já foi concluída por ele. O projeto demonstra sequência, seleção, repetição e modularização em JavaScript e SQL, sem exigir uma reimplementação em C que não consta da proposta fornecida.

4.1 Eixos da rubrica oficial

Disciplina exigida	Conceito aplicado	Evidência a vincular
Linguagem de Programação	Tipos, funções, eventos e entrada/saída	Componentes React e unidade funcional real
Algoritmos e Programação Estruturada	Sequência, decisão, laços e decomposição	Pseudocódigo e funções/triggers
Matemática Computacional	Somatórios, razões, percentuais e conversões	Cálculos e casos com resultados esperados

4.2 Aprofundamento nas disciplinas atuais

Disciplina atual de Gabriel	Aplicação	Produto acadêmico
Desenvolvimento em JavaScript	Estado, componentes, validação e chamadas à API	Explicação do fluxo e código correspondente
Computação em Nuvem	BaaS, serviços gerenciados e controle de acesso	Arquitetura e plano de implantação
Análise Orientada a Objetos	Entidades, responsabilidades e relações	Modelo conceitual e caso de uso

A matriz preserva os três eixos da rubrica e acrescenta as disciplinas atuais. A aceitação dessa adequação curricular cabe ao docente. Para Rafaela, no 1º semestre, a proposta de participação concentra levantamento do problema, fluxo de atendimento, conferência matemática e documentação, sem presumir sua grade completa.

5 Modelagem algorítmica

5.1 Entrada processamento e saída

O fluxo recebe unidade, cliente, tipo de atendimento, variação do produto, quantidade e adicionais. O processamento verifica os dados, registra a composição do pedido, consulta a ficha técnica e calcula o consumo. A saída é o registro da venda com seus itens e a movimentação correspondente no estoque da loja. O retorno da API informa sucesso ou falha à interface.

5.2 Decomposição do problema

A interface identifica o cliente pelo CPF. Quando não o encontra, abre o cadastro com o identificador preenchido e, após salvar, retoma o pedido. O componente de endereço consulta o ViaCEP, com possibilidade de correção manual. A regra de atendimento impede pedido presencial em unidade configurada exclusivamente para delivery.

O banco concentra automações descritas para itens de pedido e adicionais. A inserção de um item aciona a baixa conforme a ficha técnica. O pseudocódigo abaixo representa essa regra de forma conceitual; os nomes dos procedimentos não afirmam a existência de funções com esses identificadores no schema.

```
PROCEDIMENTO processar_item(item, loja)
  validar_quantidade_e_variacao(item)
  validar_tipo_atendimento(loja, pedido)
  ficha <- consultar_ficha(item.variacao)
  PARA CADA componente EM ficha
    porcao_base <- converter_unidade(componente)
    consumo <- item.quantidade * porcao_base
    baixar_estoque(loja, componente.ingrediente, consumo)
  FIM PARA
  processar_adicionais(item)
  RETORNAR resultado_da_operacao
FIM PROCEDIMENTO
```

5.3 Regras de consistência a validar

Sequência aparece nas etapas de validação e registro; seleção, nas regras de atendimento e entradas inválidas; repetição, na lista de ingredientes; modularização, nos procedimentos especializados. O teste deve conferir atomicidade da gravação, concorrência e comportamento de cancelamentos. Reenvio de uma requisição não deve provocar baixa duplicada. Esses controles são critérios de aceitação, ainda não resultados comprovados.

6 Modelagem matemática

As regras quantitativas participam do funcionamento da aplicação. Seja q_i a quantidade de uma variação, p_i seu preço unitário em reais, c_i seu custo unitário e a_{ij} a porção do ingrediente j na unidade de estoque. O custo calculado por ficha é um custo teórico; perdas reais e atualizações de compra exigem controles adicionais.

Medida	Regra e unidade
Subtotal dos itens	$S = \text{soma de } q_i \times p_i, \text{ em R\$}$
Total cobrado	$T = S - \text{desconto} + \text{taxa de entrega}, \text{ em R\$}$
Custo teórico	$C = \text{soma de } q_i \times c_i, \text{ em R\$}$
Consumo do ingrediente j	$Q_j = \text{soma de } q_i \times a_{ij}, \text{ em kg, g ou unidade}$
CMV do produto	$c \div p \times 100, \text{ em \%}, \text{ para } p > 0$
Margem bruta simplificada	$p - c, \text{ em R\$}; (p - c) \div p \times 100, \text{ em \%}$
Ticket médio	$\text{Faturamento elegível} \div \text{pedidos elegíveis}$

6.1 Exemplo verificável de custo e margem

Para um produto com preço de R\$ 27,90 e custo de ficha de R\$ 8,90, o CMV é $8,90 \div 27,90 \times 100 = 31,90\%$. A margem bruta simplificada é R\$ 19,00, equivalente a 68,10%. Os percentuais usam arredondamento a duas casas. Essa margem não é lucro líquido, pois não deduz despesas operacionais, tributos e taxas.

6.2 Exemplo de conversão e consumo

Um pedido de três burgers com porção de carne de 200 g consome $3 \times 200 = 600$ g, equivalentes a 0,60 kg. Com estoque inicial hipotético de 5,00 kg, o saldo esperado após a baixa é 4,40 kg. Porções e estoque precisam usar unidades compatíveis antes da subtração.

6.3 Critérios dos indicadores

Taxas de plataforma, pagamento e entrega devem ter tratamento explícito. Para CMV agregado, definir primeiro a receita usada no denominador; o total cobrado ao cliente não deve ser adotado automaticamente como receita dos produtos. O ticket precisa do mesmo período, lojas e estados de pedido no numerador e denominador. Divisões por zero retornam indicador não calculável, sem erro ou valor infinito.

7 Análise orientada a objetos

A modelagem conceitual organiza os objetos do domínio e suas responsabilidades. A UML fornece uma linguagem padronizada para representar essas relações (OBJECT MANAGEMENT GROUP, 2017). O modelo abaixo explica o sistema sem afirmar que o front-end emprega classes JavaScript: componentes funcionais React podem representar interfaces relacionadas aos mesmos objetos.

Objeto	Responsabilidade	Associação conceitual
Cliente	Identidade e contato do comprador	1 cliente possui 0..* pedidos
Pedido	Venda, estado, canal e totais	1 pedido contém 1..* itens
ItemPedido	Quantidade e preço praticado	Cada item refere 1 variação
Produto	Identidade comercial e categoria	1 produto possui 1..* variações
Variação	Tamanho e composição vendável	Ficha relaciona variação a ingredientes
Loja	Unidade operacional	1 loja possui 0..* pedidos

7.1 Caso de uso principal

Caso de uso: registrar pedido. Ator principal: operador. Pré-condições: sessão válida, permissão para operar a unidade e catálogo disponível. Fluxo principal: selecionar loja, identificar cliente, selecionar variações e adicionais, conferir valores, registrar o pedido e receber a confirmação. Pós-condição esperada: pedido e itens persistidos, com efeitos de estoque coerentes com a regra adotada.

Fluxos alternativos: cliente inexistente leva ao cadastro e retorno ao pedido; CPF ou quantidade inválidos exigem correção; indisponibilidade do ViaCEP permite informar endereço manualmente; atendimento incompatível com a loja impede a operação; falha de gravação deve ser comunicada sem confirmação indevida.

7.2 Encapsulamento das responsabilidades

As telas cuidam da interação. O acesso a dados utiliza a API do Supabase. As regras centrais descritas ficam no banco. Essa divisão evita depender apenas de validações visuais, que podem ser contornadas por uma chamada direta à API. A fronteira entre responsabilidades deve permanecer identificável no código e nas migrações.

8 Banco de dados e camada analítica

A ficha técnica descreve mais de vinte tabelas, incluindo lojas, categorias, produtos, produto_variacoes, ingredientes, fichas_tecnicas, clientes, pedidos, itens_pedido, estoque_lojas, fornecedores, usuarios, perfis e permissoes. Também registra tabelas para adicionais, cervejas, torneiras de chope e configurações de canal e pagamento.

8.1 Automação transacional

Os triggers descritos baixam ingredientes de itens e adicionais, validam o tipo de atendimento e criam registro em usuarios após um novo login. Funções também sustentam cálculos de troco e taxas. Cada automação deve ser documentada com evento de disparo, tabelas afetadas, condições e estratégia para reversão ou compensação.

8.2 Consultas e dimensões

Objeto informado	Finalidade
vw_custo_variacao	Custo e margem da variação
vw_vendas_por_loja	Faturamento e ticket por unidade
vw_ranking_produtos	Classificação dos produtos vendidos
vw_engenharia_cardapio	Leitura de popularidade e rentabilidade
dim_loja, dim_produto, dim_data e outras	Contexto de análise por entidade e período
fato_vendas e fato_itens_venda	Medidas em níveis de pedido e item

Uma view comum guarda a definição da consulta e a executa quando referenciada; não armazena fisicamente o resultado como uma materialized view (POSTGRESQL GLOBAL DEVELOPMENT GROUP, s.d.). Assim, a expressão “consulta materializada em tempo real” não define corretamente todas as views do projeto. O tipo de cada objeto deve ser confirmado nas migrações.

8.3 Consistência analítica

A análise por item não pode multiplicar o total do pedido pelo número de linhas. Taxas e descontos precisam de uma regra de rateio quando forem levados ao nível do item. Preservar preço e custo de referência da venda evita que uma alteração posterior do cadastro reescreva a interpretação histórica. A existência de dimensões e fatos prepara a integração, mas não comprova Power BI conectado ou ETL produtivo.

9 Computação em nuvem e segurança

Supabase fornece o backend gerenciado utilizado pelo projeto. A interface permanece em desenvolvimento local com Vite; o Netlify é a hospedagem planejada para o front-end. Não se pressupõem implantação contínua, política de backup ou capacidade contratada sem configurar e verificar o ambiente correspondente.

9.1 Fluxo de acesso

O navegador autentica o usuário, mantém a sessão e solicita operações à API do Supabase. O PostgREST encaminha o acesso ao PostgreSQL, onde permissões e políticas condicionam o resultado. Os scripts Python fazem operações independentes de preparação e simulação. Serviços externos, como ViaCEP, atendem funções específicas da interface.

Controle	Função	Verificação necessária
Supabase Auth	Identificar usuário e sessão	Login, expiração e encerramento
RBAC	Relacionar perfil e permissões	Ações permitidas por papel
RLS	Controlar linhas acessíveis	Leitura e escrita entre lojas
Funções privilegiadas	Executar automações de sistema	Privilégios mínimos e chamadas autorizadas

RLS aplica condições de acesso dentro do PostgreSQL (SUPABASE, s.d.b). A ficha informa políticas com escopo por unidade em algumas tabelas. Ativar RLS não prova que todas as combinações de usuário, loja e ação estão protegidas. Também é necessário revisar o acesso às views e às funções SECURITY DEFINER para que seus privilégios não permitam operações indevidas.

9.2 Configuração e operação

Senhas do banco e chaves privilegiadas não devem entrar no código do front-end nem no repositório. O arquivo de exemplo de ambiente registra nomes e valores fictícios. A documentação de implantação deverá indicar variáveis, configuração de autenticação, migrações, logs e procedimento de recuperação conforme os recursos efetivamente disponíveis.

Antes da publicação, validar o comportamento em ambiente de teste com dados sintéticos. O acesso entre unidades, cancelamentos e requisições repetidas fazem parte dos testes necessários para uma operação consistente.

10 Plano de implementação e validação

A implementação deve evoluir em entregas pequenas. A sequência proposta aproveita o sistema existente e prioriza a possibilidade de reproduzir e verificar seu funcionamento antes de expandir módulos.

Etapa	Entrega	Critério de conclusão
1 Consolidar	Código, README e migrations no Git	Ambiente inicial reproduzível
2 Validar	Cálculos, estoque e autorização	Resultados registrados e falhas corrigidas
3 Integrar compras	Custo de compra nos ingredientes	Ficha usa custo de referência coerente
4 Publicar	Front-end e rotina de implantação	Ambiente acessível e configurado
5 Evoluir	Financeiro e Power BI	Dados reconciliados com a operação

10.1 Casos iniciais de teste

Caso	Entrada ou situação	Resultado esperado
CT01	Preço 27,90; custo 8,90	CMV 31,90%; margem R\$ 19,00
CT02	3 burgers de 200 g; saldo 5 kg	Baixa 0,60 kg; saldo 4,40 kg
CT03	Quantidade zero ou negativa	Rejeição sem baixa de estoque
CT04	Presencial em loja delivery-only	Rejeição da operação
CT05	Usuário acessa unidade não autorizada	Negação conforme política
CT06	Reenvio ou cancelamento do pedido	Sem baixa duplicada ou saldo incoerente
CT07	Receita ou preço igual a zero	Percentual não calculável, sem divisão inválida

Os resultados acima são expectativas de teste. A ficha técnica informa que a camada automatizada ainda precisa ser criada. Os testes do módulo demonstrativo entregue anteriormente não comprovam a aplicação real. A evidência de execução deverá identificar versão do código, ambiente, comando, dados utilizados e resultado obtido.

11 Repositório e organização da equipe

O Git ainda não está formalizado conforme a ficha técnica. Por isso, não há URL e histórico verificados para registrar como concluídos nesta revisão. Antes da submissão, o grupo deve publicar ou compartilhar o repositório com o docente, inserir seu endereço completo no trabalho e testar o acesso fora da conta do responsável.

11.1 Estrutura proposta para organização

A estrutura abaixo é uma proposta de organização. Ela deve ser adaptada aos caminhos reais da aplicação, preservando arquivos de configuração e dependências necessários à execução.

Pasta ou arquivo	Conteúdo previsto
README.md	Tema, integrantes, problema, tecnologias e execução
src/	Componentes, páginas e acesso ao Supabase
supabase/migrations/	Schema, views, funções, triggers e políticas
scripts/	Seeds e simulações Python
tests/	Casos unitários e de integração
docs/entrega-01/	Trabalho, apresentação, modelagens e evidências
Manifestos e arquivo de ambiente de exemplo	Dependências reais e configuração sem segredos

11.2 Histórico e contribuições

O primeiro commit deve identificar o estado recebido como linha de base. Alterações posteriores precisam de mensagens descritivas e correspondência com o trabalho realizado. Não se devem fabricar datas, dividir artificialmente código pronto para simular evolução ou atribuir autoria que não corresponda à contribuição efetiva.

A divisão proposta atribui a Gabriel a revisão técnica de JavaScript, arquitetura Supabase, modelagem OO e regras de banco. Rafaela participa do levantamento do fluxo, descrição do problema, conferência matemática, casos de teste e revisão textual. Essa organização é um plano de trabalho; os registros reais devem confirmar as contribuições.

11.3 Pacote acadêmico

A proposta oficial exige PI-2026-2-BRASA-BURGUER-ENTREGA-1.zip, até 100 MB, com trabalho-escrito.pdf, apresentacao.pdf e anexos/. O formulário recebe título, URL do repositório e ZIP. O prazo informado é 02/10/2026, respeitando o horário institucional. Código e evidências devem refletir o mesmo estado da documentação.

12 Perspectivas e considerações finais

12.1 Perspectiva de produto

O posicionamento proposto é um sistema especializado em food service, inicialmente direcionado a hamburguerias independentes e pequenas redes. O valor esperado está na associação entre pedido, ficha técnica, consumo e desempenho por unidade. Essa hipótese deve ser validada em um piloto, observando facilidade de uso, consistência dos números e esforço operacional.

Os planos Starter, Pro, Business e Enterprise são possibilidades comerciais descritas na apresentação, sem preços definidos. Assinatura, implantação, treinamento, migração e serviços de dados podem compor o modelo de receita. Não representam vendas realizadas nem validação de mercado. A evolução comercial depende de maturidade técnica, suporte e aderência às necessidades dos usuários.

12.2 Integrações posteriores

O roadmap inclui marketplaces, WhatsApp, gateways de pagamento, emissão fiscal, Power BI e ferramentas de inteligência artificial. Essas integrações devem ser tratadas como futuras. Um copiloto baseado em linguagem natural e modelos de previsão exigem dados confiáveis, regras de autorização e critérios de avaliação antes de participar das decisões da operação.

12.3 Síntese acadêmica

O Brasa Burguer Management reúne um caso de programação e modelagem com aplicação prática. A interface utiliza React, a API é fornecida pelo Supabase e as regras descritas concentram-se no PostgreSQL. Python apoia preparação e simulação de dados de forma independente. A arquitetura atual permite relacionar o projeto às disciplinas de JavaScript, nuvem e análise orientada a objetos, preservando os eixos de algoritmos e matemática exigidos pela rubrica.

A Entrega 1 estabelece problema, escopo, continuidade e modelos verificáveis. O foco imediato é vincular as funcionalidades documentadas ao código original, aos testes e ao histórico Git. Essa etapa permite que o docente diferencie a linha de base, o incremento de 2026/2 e as evoluções futuras, sem confundir descrição de produto com comprovação de implementação.

Referências

ANHANGUERA. Projeto Integrado I, II e III: Entrega 1 – Proposta e modelagem integrada. Material da disciplina, Prof. Gilberto Falco Netto, 2026/2. Documento fornecido ao grupo.

BRASA BURGUER. Ficha Técnica do Projeto. Documento técnico do projeto, 2026. Material fornecido para esta revisão.

BRASA BURGUER. Apresentação Executiva. Arquivo Brasa_Burguer_Apresentacao_Executiva(1).pptx, 2026. Material fornecido para esta revisão.

OBJECT MANAGEMENT GROUP. Unified Modeling Language, version 2.5.1. 2017. Disponível em: <https://www.omg.org/spec/UML/2.5.1>. Acesso em: 13 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. CREATE VIEW. PostgreSQL Documentation. s.d. Disponível em: <https://www.postgresql.org/docs/current/sql-createview.html>. Acesso em: 13 set. 2026.

PYTHON SOFTWARE FOUNDATION. The Python Tutorial. s.d. Disponível em: <https://docs.python.org/3/tutorial/>. Acesso em: 13 set. 2026. Apoio à decomposição, estruturas de controle e scripts.

REACT. Thinking in React. s.d. Disponível em: <https://react.dev/learn/thinking-in-react>. Acesso em: 13 set. 2026.

ROSEN, Kenneth H. Discrete Mathematics and Its Applications. 8. ed. McGraw-Hill, 2019. Referência de apoio para relações, funções e modelagem matemática.

SUPABASE. Data REST API. s.d.a. Disponível em: <https://supabase.com/docs/guides/api>. Acesso em: 13 set. 2026.

SUPABASE. Row Level Security. s.d.b. Disponível em: <https://supabase.com/docs/guides/database/postgres/row-level-security>. Acesso em: 13 set. 2026.